

Skill - 7

1. Path command Resolution

```
harshika@siri:~$ gedit skill7.c
** (gedit:2433): WARNING **: 04:03:14.013: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found
** (gedit:2433): WARNING **: 04:03:14.015: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found
harshika@siri:~$ gcc skill7.c -o skill7
harshika@siri:~$ ./skill7

=====
SKILL 7 - PROCESS & PATH
=====

PART 1: PROCESS SYNCHRONIZATION
=====
Parent PID: 2616

[PARENT PROCESS]
Created child with PID: 2617
Parent is waiting for child...

[CHILD PROCESS]
Child PID: 2617
Child is performing some work...
Child work completed.
Parent detected that child has finished.
Child exit status: 0

=====
PART 2: PATH VARIABLE
=====
PATH variable:
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/usr/lib/mpi/lib:/mnt/c/Program Files/WindowsApps/Microsoft Corporation/WindowsSubsystemForLinux_2.7.11.0_x64__8wekyb3d8bbwe:/mnt/c/Program Files/Common Files/Oracle/Java/javapath:/mnt/c/Program Files (x86)/Common Files/Oracle/Java/javapath:/mnt/c/Program Files (x86)/Common Files/Oracle/Java/javapath:/mnt/c/WINDOWS/system32:/mnt/c/WINDOWS:/mnt/c/WINDOWS/System32/Wbem:/mnt/c/WINDOWS/System32/WindowsPowerShell/v1.0:/mnt/c/WINDOWS/System32/OpenSSH:/mnt/c/jdk-25_windows-x64_bin.zip/jdk-25:/mnt/c/WINDOWS/system32/c
```

```
1#include <stdio.h>
2#include <stdlib.h>
3#include <string.h>
4#include <unistd.h>
5#include <sys/types.h>
6#include <sys/wait.h>
7#include <sys/stat.h>
8#include <errno.h>
9
10#define MAX_PATH 4096
11#define MAX_COMMAND 100
12
13
14/*
15 PART 1: PROCESS SYNCHRONIZATION USING waitpid()
16 ===== */
17
18void demonstrate_waitpid()
19{
20     printf("\n\n\n");
21     printf(" PART 1: PROCESS SYNCHRONIZATION\n");
22     printf("===== \n");
23
24     printf("Parent PID: %d\n", getpid());
25
26     pid_t pid = fork();
27
28     if (pid < 0)
29     {
30         perror("fork failed");
31         return;
32     }
33
34     if (pid == 0)
35     {
36         /* Child process */
37         printf("\n[CHILD PROCESS]\n");
38         printf("Child PID: %d\n", getpid());
39
40         printf("Child is performing some work...\n");
41         sleep(3);
42
43         printf("Child work completed.\n");
44
45         exit(0);
46     }
47     else
48     {
49         /* Parent process */
50         int status;
51
52         printf("\n[PARENT PROCESS]\n");
53     }
54 }
```